Local Search

Optimization Problems

- Brute force search
  - Backtracking
- Branch and Bound
- Local Search
- Dynamic programming
- Divide and Conquer
- Greedy

Why local search?

Issues with the search algorithms:

1) Tries to explore the entire search space systematically.

If the search space is big => long time to reach to the GOAL.

If the search space is infinite => cannot hope to visit all the states.

2) The search algorithm remembers the path from the initial state to the GOAL state. Is this really necessary?

The **4-Queens problem**: we do not need to know from which state we obtain the GOAL state.

Why local search?

Issues with the search algorithms:

1) Tries to explore the entire search space systematically.

If the search space is big => long time to reach to the GOAL.

If the search space is infinite => cannot hope to visit all the states.

Local search does not explore the search space systematically.

2) The search algorithm remembers the path from the initial state to the GOAL state. Is this really necessary?

The **4-Queens problem**: we do not need to know from which state we obtain the GOAL state.

Local search does not keep track of the path from initial state to GOAL state. It only remembers the inital state.

Properties of Local Search

1) **Local search does not explore the search space systematically.**

2) **Local search does not keep track of the path from initial state to GOAL state. It only remembers the inital state.**

Properties of Local Search (LS)

1) **Local search does not explore the search space systematically.**

Instead of exploring all possible states, LS uses strategies to find reasonably good states as quickly as possible on average.

Good enough in practice!

Disadvantage:

Not guaranteed to find a solution (even if a solution exists).

Can we use this to prove no solution exists?

Therefore, LS only when a solution exists!

Properties of Local Search

2) **Local search does not keep track of the path from initial state to GOAL state. It only remembers the inital state.**

Therefore local search algorithms require very little memory since they only need to remember the initial state.

Components of a LS problem

A local search problem consists of

- Complete state (initial): decision on all the items/variables of the problem.
- Neighbor relation: what states can I reach from the initial state?
- Cost function: how good is each state?

Components of a LS problem

A local search problem consists of

- Complete state (initial): decision on all the items/variables of the problem.
- Neighbor relation: what states can I reach from the initial state?
- Cost function: how good is each state?

The n-Queens problem

- Complete state:
- Neighbor relation: what strategies?
- Cost function:

Components of a LS problem

A local search problem consists of

- Complete state (initial): decision on all the items/variables of the problem.
- Neighbor relation: what states can I reach from the initial state?
- Cost function: how good is each state?

The n-Queens problem

- Complete state: any complete state.
  Goal state: no two Queen attack each other
- Neighbor relation: what strategies?
- Cost function:

Components of a LS problem

A local search problem consists of

- Complete state (initial): decision on all the items/variables of the problem.
- Neighbor relation: what states can I reach from the initial state?
- Cost function: how good is each state?

The n-Queens problem

- Complete state: any complete state.
  Goal state: no two Queen attack each other
- Neighbor relation: what strategies?

  Strategy A: move a queen to a different row in the same coulmn
  Strategy B: swap the row positions of two queens.

- Cost function:

Components of a LS problem

A local search problem consists of

- Complete state (initial): decision on all the items/variables of the problem.
- Neighbor relation: what states can I reach from the initial state?
- Cost function: how good is each state?

The n-Queens problem

- Complete state: any complete state.
  Goal state: no two Queen attack each other
- Neighbor relation: what strategies?

  Strategy A: move a queen to a different row in the same coulmn
  Strategy B: swap the row positions of two queens.

- Cost function: number of pairs of queens attacking each other.
  (directly or indirectly)

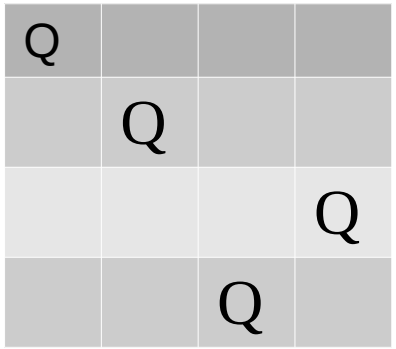
Components of a LS problem

The n-Queens problem

- Complete state: any complete state.
  Goal state: no two Queen attack each other
- Neighbor relation: what strategies?

  Strategy A: move a queen to a different row in the same coulmn
  Strategy B: swap the row positions of two queens.

- Cost function: number of pairs of queens attacking each other.
  (directly or indirectly)



Cost of this object? 2

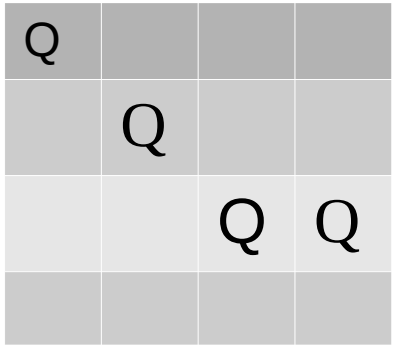
Components of a LS problem

The n-Queens problem

- Complete state: any complete state.
  Goal state: no two Queen attack each other
- Neighbor relation: what strategies?

  Strategy A: move a queen to a different row in the same coulmn
  Strategy B: swap the row positions of two queens.

- Cost function: number of pairs of queens attacking each other.
  (directly or indirectly)



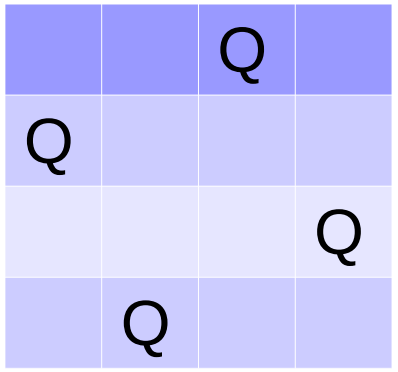
Cost of this object? 4

Components of a LS problem

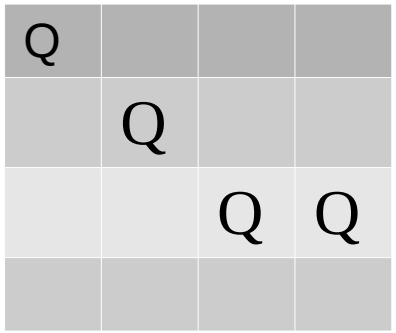
Neighbor relation:

Strategy A: move a queen to a different row in the same coulmn.

Strategy B: swap the row positions of two queens.

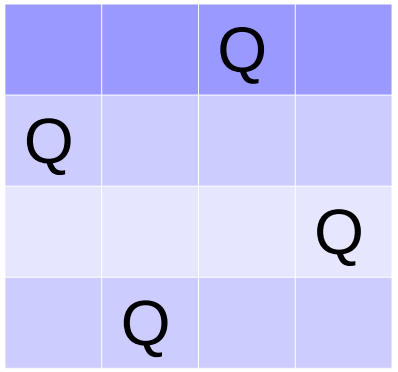


Goal State



Does the initial state require to satisfy the constraints?

Components of a LS problem



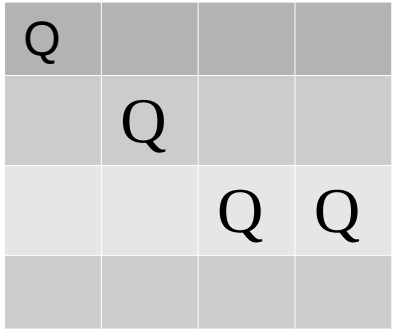
Goal State

Neighbor relation:

Strategy A: move a queen to a different row in the same coulmn.

Strategy B: swap the row positions of two queens.

S_0 -> S_1 -> S_2 -> ..... -> S_f



Does the initial state require to satisfy the constraints?

Greedy Descent (LS algorithm)

- Start with a random state
- At each step, decide if we want to go to a neighboring state.
- When do we go?

Greedy Descent (LS algorithm)

- Start with a random state
- At each step, decide if we want to go to a neighboring state.
- When do we go? If the neighbor has a cost strictly less than the cost of the current state.
- Choose a neighbor who cost is minimum (arbitarily pick one if there are multiple choices)
- What happens if movement is not possible?

Greedy Descent (LS algorithm)

- Start with a random state
- At each step, decide if we want to go to a neighboring state.
- When do we go? If the neighbor has a cost strictly less than the cost of the current state.
- Choose a neighbor who cost is minimum (arbitarily pick one if there are multiple choices)
- What happens if movement is not possible? **The current state is the best state in its neighborhood.**

Also called “Hill Climbing”

Properties of Greedy Descent (GD)

- Looks simple and it is known to be produce good results in practice.
- If GD has unlimited time to explore the search space, will be reach to an optimal solution?

Properties of Greedy Descent (GD)

- Looks simple and it is known to be produce good results in practice.
- If GD has unlimited time to explore the search space, will be reach to an optimal solution?
    - It stops at the point where it is the best state in its neighborhood. (Local Optimum)

Properties of Greedy Descent (GD)

- Looks simple and it is known to be produce good results in practice.
- If GD has unlimited time to explore the search space, will be reach to an optimal solution?
    - It stops at the point where it is the best state in its neighborhood. (Local Optimum)

Global optimum means a state with the lowest cost among all the states in the search space.

Best state in its local neighborhood **vs** Best state in the **state space**.

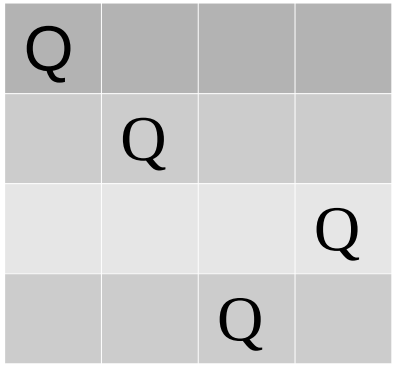
Local Optimum

Consider the following state of the 4-queens problem. Consider neighbour relation: **swap the row positions of two queens**. Which of the following is correct?

a) This state is a local optimum and is a global optimum.

b) This state is a local optimum and is not a global optimum.

c)This state is not a local optimum and is not a global optimum.



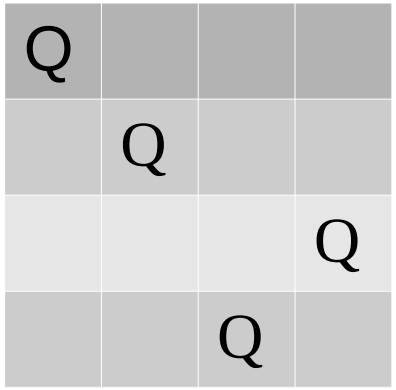
Local Optimum

Consider the following state of the 4-queens problem. Consider neighbour relation: **swap the row positions of two queens**. Which of the following is correct?

a) This state is a local optimum and is a global optimum.

b) This state is a local optimum and is not a global optimum.

c)This state is not a local optimum and is not a global optimum.



Configuration: <1, 2, 4, 3>

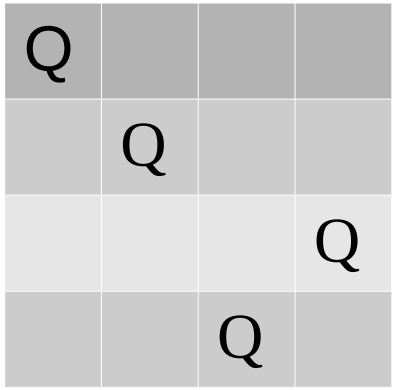
Local Optimum

Consider the following state of the 4-queens problem. Consider neighbour relation: **move a single queen to another square in the same column**. Which of the following is correct?

a) This state is a local optimum and is a global optimum.

b) This state is a local optimum and is not a global optimum.

c)This state is not a local optimum and is not a global optimum.



Configuration: <1, 2, 4, 3>

How to cope with the Local Optimums?

Maybe progress to a neighbor with the same cost with the hope that it will guide to the GOAL state?

- What if this does not happen? Flat Optimum.
- It may happen? Shoulder Optimum.

Any issues with this?

How to cope with the Local Optimums?

Maybe progress to a neighbor with the same cost with the hope that it will guide to the GOAL state?

- What if this does not happen? Flat Optimum.
- It may happen? Shoulder Optimum.

Moving sideways.
Maintaining Table.

How to cope with the Local Optimums?

Maybe progress to a neighbor with the same cost with the hope that it will guide to the GOAL state?

- What if this does not happen? Flat Optimum.
- It may happen? Shoulder Optimum.

Moving sideways.
Maintaining Table.

One strategy:
GD with Random moves.